NBA Future Analytics Stars Tech Challenge

Isaac Yu

2025-12-01

This project was completed for the NBA's Rising Stars Analyst Program. It attempts to predict the 2025-2026 Rookie of the Year from historical data.

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(stringi)
library(stringr)
library(ggplot2)
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(gridExtra)
```

```
##
## Attaching package: 'gridExtra'
```

```
## The following object is masked from 'package:dplyr':
##
##     combine
```

```
library(caret)
```

```
## Loading required package: lattice
```

```r
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-10
```

```r
library(ROCR)
library(e1071)
```

```
##
## Attaching package: 'e1071'
```

```
## The following object is masked from 'package:ggplot2':
##
##     element
```

```r
library(randomForest)
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
## Attaching package: 'randomForest'
```

```
## The following object is masked from 'package:gridExtra':
##
##     combine
```

```
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```
## The following object is masked from 'package:dplyr':
##
##     combine
```

LOAD DATA

```r
#Basic stats for previous & current rookies
prev_rooks <- read.csv("Previous_Rookies.csv")
current_rooks <- read.csv("Current_Rookies.csv")

#Advanced stats for previous & current rookies
prev_rooks_adv <- read.csv("Previous_Rookies_ADV.csv")
current_rooks_adv <- read.csv("Current_Rookies_ADV.csv")
```

DATA CLEANING

```
#Check number of NAs in each dataset
cat("There are", sum(is.na(prev_rooks)), "NA values in prev_rooks.\n")
```

```
## There are 0 NA values in prev_rooks.
```

```
cat("There are", sum(is.na(current_rooks)), "NA values in current_rooks.\n")
```

```
## There are 0 NA values in current_rooks.
```

```
cat("There are", sum(is.na(prev_rooks_adv)), "NA values in prev_rooks_adv.\n")
```

```
## There are 0 NA values in prev_rooks_adv.
```

```
cat("There are", sum(is.na(current_rooks_adv)), "NA values in current_rooks_adv.\n")
```

```
## There are 0 NA values in current_rooks_adv.
```

No need for any imputations.

```
#Turn categorical variables into factors

prev_rooks <- prev_rooks %>%
  mutate(
    #Remove accents
    Player = stri_trans_general(Player, "Latin-ASCII"),
    Player = make.names(Player),
    Team = make.names(Team),
    SEASON = as.factor(SEASON),
    ROY_winner = factor(ROY_winner, levels = c(0,1), labels = c("No","Yes"))
  )

prev_rooks_adv <- prev_rooks_adv %>%
  mutate(
    #Remove accents
    PLAYER = stri_trans_general(PLAYER, "Latin-ASCII"),
    PLAYER = make.names(PLAYER),
    TEAM = make.names(TEAM)
  )

current_rooks <- current_rooks %>%
  mutate(
    Team = make.names(Team),
    SEASON = as.factor(SEASON),
  )

current_rooks_adv <- current_rooks_adv %>%
  mutate(
    TEAM = make.names(TEAM)
  )
```

Only selected seasons 2015-2025 as testing, removing any special characters/accent marks from names for modelling.

```
#Merge data
prev_full <- prev_rooks %>%
  left_join(prev_rooks_adv, by = c("Player" = "PLAYER", "Team" = "TEAM"))

current_full <- current_rooks %>%
  left_join(current_rooks_adv, by = c("Player" = "PLAYER", "Team" = "TEAM"))

cat("There are", sum(is.na(prev_full)), "NA values in prev_full.\n")
```

```
## There are 0 NA values in prev_full.
```

```
cat("There are", sum(is.na(current_full)), "NA values in current_full.\n")
```

```
## There are 0 NA values in current_full.
```

Merge prev_rooks & prev_rooks_adv for FULL stats, and current_rooks & current_rooks_adv for FULL stats.

FEATURE SELECTION

```
#Drop columns with the same data (prefer per-game stats than % stats)
cat("There are", ncol(prev_full) - 1, "predictors in the data:\n")
```

```
## There are 47 predictors in the data:
```

```
names(prev_full)[-32]
```

```
##  [1] "Player" "Team"   "Age"    "GP"     "W"      "L"      "Min"    "PTS"
##  [9] "FGM"    "FGA"    "FG."    "X3PM"   "X3PA"   "X3P."   "FTM"    "FTA"
## [17] "FT."    "OREB"   "DREB"   "REB"    "AST"    "TOV"    "STL"    "BLK"
## [25] "PF"     "FP"     "DD2"    "TD3"    "X..."   "SEASON" "PCT_GP" "OFFRTG"
## [33] "DEFRTG" "NETRTG" "AST."   "ASTTO"  "ASTRAT" "OREB."  "DREB."  "REB."
## [41] "TORAT"  "EFG."   "TS."    "USG."   "PACE"   "PIE"    "POSS"
```

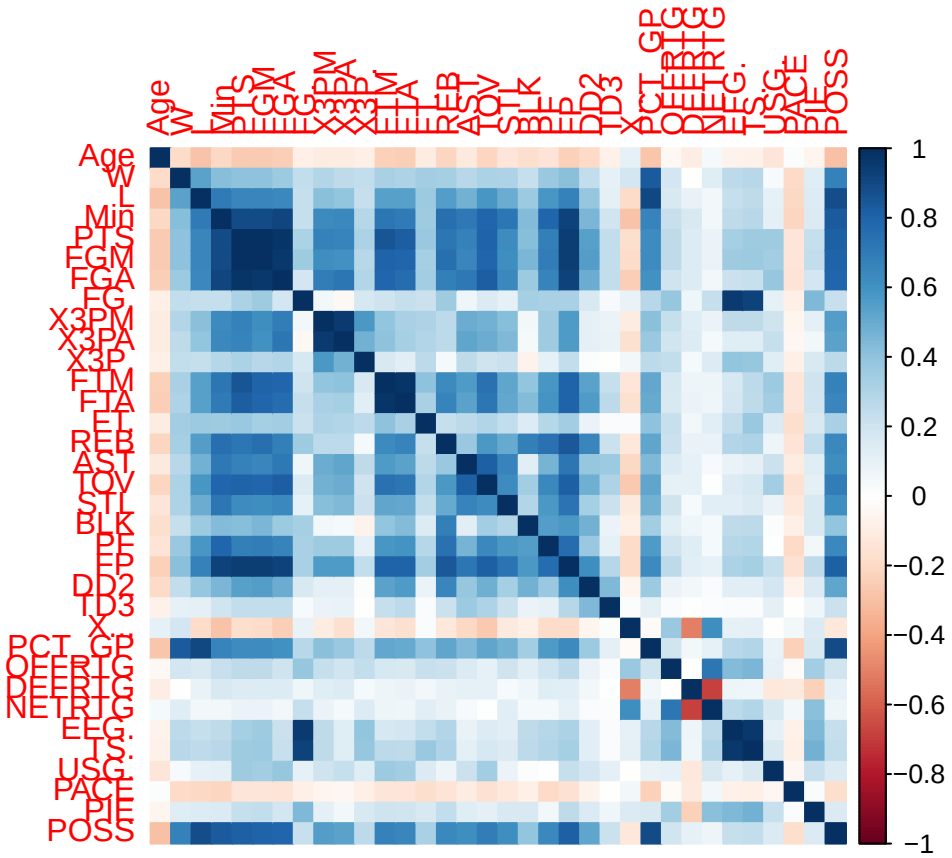
```
prev_full <- prev_full %>%
  select(-c(GP, #Corresponds to PCT_GP
            OREB, DREB, #Linear combination of REB
            AST., ASTTO, ASTRAT, #Corresponds to AST
            OREB., DREB., #Linear combination of REB.
            REB., #Corresponds to REB
            TORAT #Corresponds to TOV
            )
         )

cat("After dropping similar predictors, we now have", ncol(prev_full) - 1, "predictors:\n")
```

After dropping similar predictors, we now have 37 predictors:

```
names(prev_full)[-29]
```

```
##  [1] "Player" "Team"   "Age"    "W"      "L"      "Min"    "PTS"    "FGM"
##  [9] "FGA"    "FG."    "X3PM"   "X3PA"   "X3P."   "FTM"    "FTA"    "FT."
## [17] "REB"    "AST"    "TOV"    "STL"    "BLK"    "PF"     "FP"     "DD2"
## [25] "TD3"    "X..."   "SEASON" "PCT_GP" "OFFRTG" "DEFRTG" "NETRTG" "EFG."
## [33] "TS."    "USG."   "PACE"   "PIE"    "POSS"
```

```
#Find highly correlated
numeric_vars <- names(prev_full)[sapply(prev_full, is.numeric)]
numeric_data <- prev_full %>% select(all_of(numeric_vars))
corr_matrix <- cor(numeric_data, use = "pairwise.complete.obs")
corrplot::corrplot(corr_matrix, method = "color")
```




```
cutoff <- 0.8

#Find all high correlation pairs
high_pairs <- which(abs(corr_matrix) > cutoff & upper.tri(corr_matrix), arr.ind = TRUE)

high_corr_pairs <- data.frame(
  var1 = rownames(corr_matrix)[high_pairs[, 1]],
  var2 = colnames(corr_matrix)[high_pairs[, 2]],
```

```
  correlation = corr_matrix[high_pairs]
)

high_corr_pairs
```

```
##      var1   var2 correlation
## 1     Min    PTS   0.8975939
## 2     Min    FGM   0.8952851
## 3     PTS    FGM   0.9905212
## 4     Min    FGA   0.9105014
## 5     PTS    FGA   0.9722944
## 6     FGM    FGA   0.9651785
## 7    X3PM   X3PA   0.9550975
## 8     PTS    FTM   0.8514932
## 9     FGM    FTM   0.8053227
## 10    PTS    FTA   0.8276299
## 11    FTM    FTA   0.9709714
## 12    PTS    TOV   0.8058707
## 13    FGA    TOV   0.8210463
## 14    AST    TOV   0.8121092
## 15    Min     FP   0.9283799
## 16    PTS     FP   0.9453131
## 17    FGM     FP   0.9488967
## 18    FGA     FP   0.9234466
## 19    FTM     FP   0.8075203
## 20    FTA     FP   0.8021675
## 21    REB     FP   0.8482096
## 22    TOV     FP   0.8121456
## 23      W PCT_GP   0.8399113
## 24      L PCT_GP   0.9012071
## 25    FG.   EFG.   0.9519927
## 26    FG.    TS.   0.9207359
## 27   EFG.    TS.   0.9656719
## 28      L   POSS   0.8803741
## 29    Min   POSS   0.8352309
## 30    PTS   POSS   0.8107514
## 31    FGM   POSS   0.8053780
## 32     FP   POSS   0.8160083
## 33 PCT_GP   POSS   0.8926552
```

```
prev_full <- prev_full %>%
  select(-c(Min, FGA, X3PA, FTA, FP, EFG., W, L,, FG., FGM))

cat("After dropping highly correlated predictors, we now have", ncol(prev_full) - 1, "predictors:\n")
```

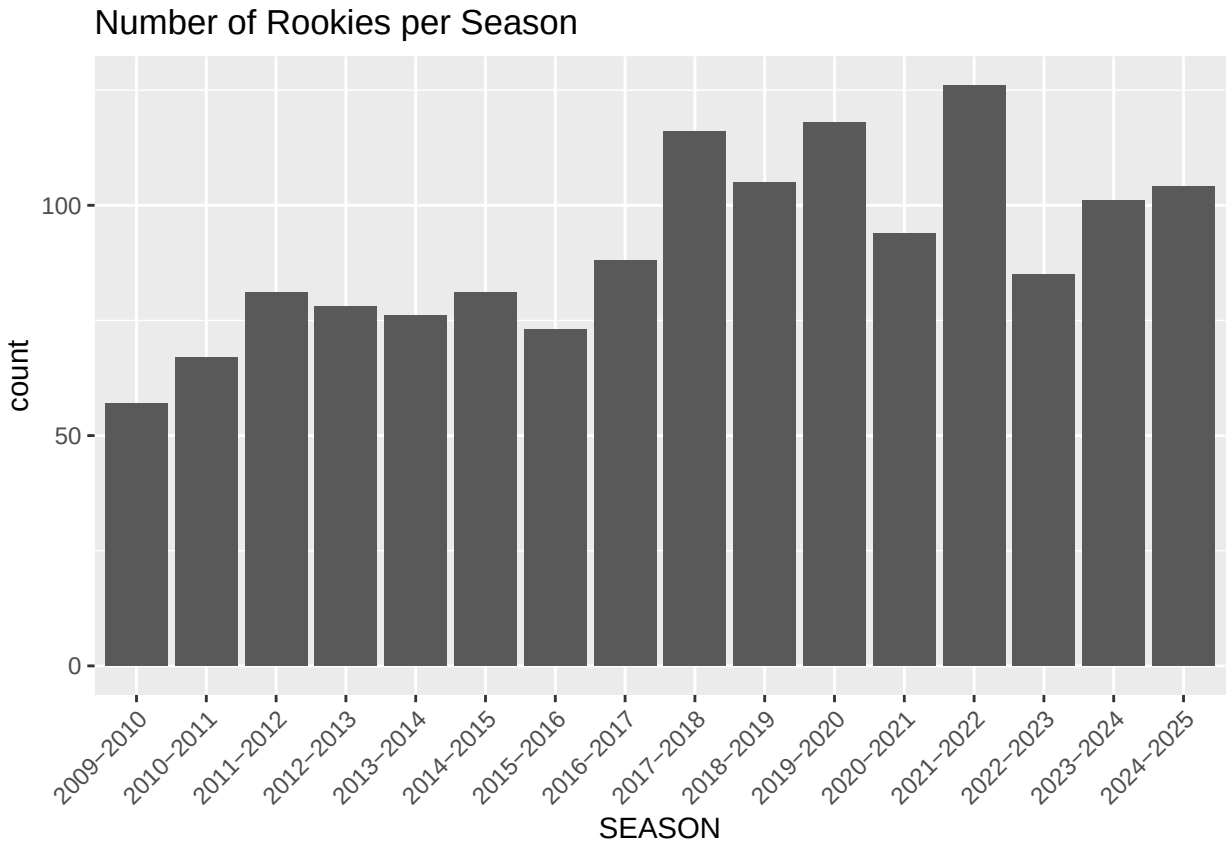
```
## After dropping highly correlated predictors, we now have 27 predictors:
```

```
names(prev_full)[-20]
```

```
##  [1] "Player" "Team"   "Age"    "PTS"    "X3PM"   "X3P."   "FTM"    "FT."
##  [9] "REB"    "AST"    "TOV"    "STL"    "BLK"    "PF"     "DD2"    "TD3"
## [17] "X..."   "SEASON" "PCT_GP" "OFFRTG" "DEFRTG" "NETRTG" "TS."    "USG."
## [25] "PACE"   "PIE"    "POSS"
```

EXPLORATORY DATA ANALYSIS

```
ggplot(prev_full, aes(SEASON)) +
  geom_bar() +
  labs(title = "Number of Rookies per Season") +
   theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



The numbers of rookies drafted per season seems to have increased slightly since the early 2010s.

```
cat("The distribution of Double-Doubles:\n")
```

```
## The distribution of Double-Doubles:
```

```
table(prev_full$DD2)
```

```
##
##    0    1    2    3    4    5    6    7    8    9   10   11   12   13   14   15
## 1064  158   65   48   23    7    7    9   13    5    3    7    5    6    4    2
##   16   17   18   19   20   21   23   24   26   30   38   39   43   52   63
##    4    1    2    2    2    3    1    1    2    1    1    1    1    1    1
```

```r
cat("The distribution of Triple-Doubles:\n")
```

```
## The distribution of Triple-Doubles:
```

```r
table(prev_full$TD3)
```

```
##
##    0    1    2    4    8   12
## 1423   17    7    1    1    1
```

```r
prev_full <- prev_full %>%
  select(-c(DD2, TD3))

cat("After removing Double-Doubles & Triple-Doubles, we now have", ncol(prev_full) - 1, "predictors:\n")
```

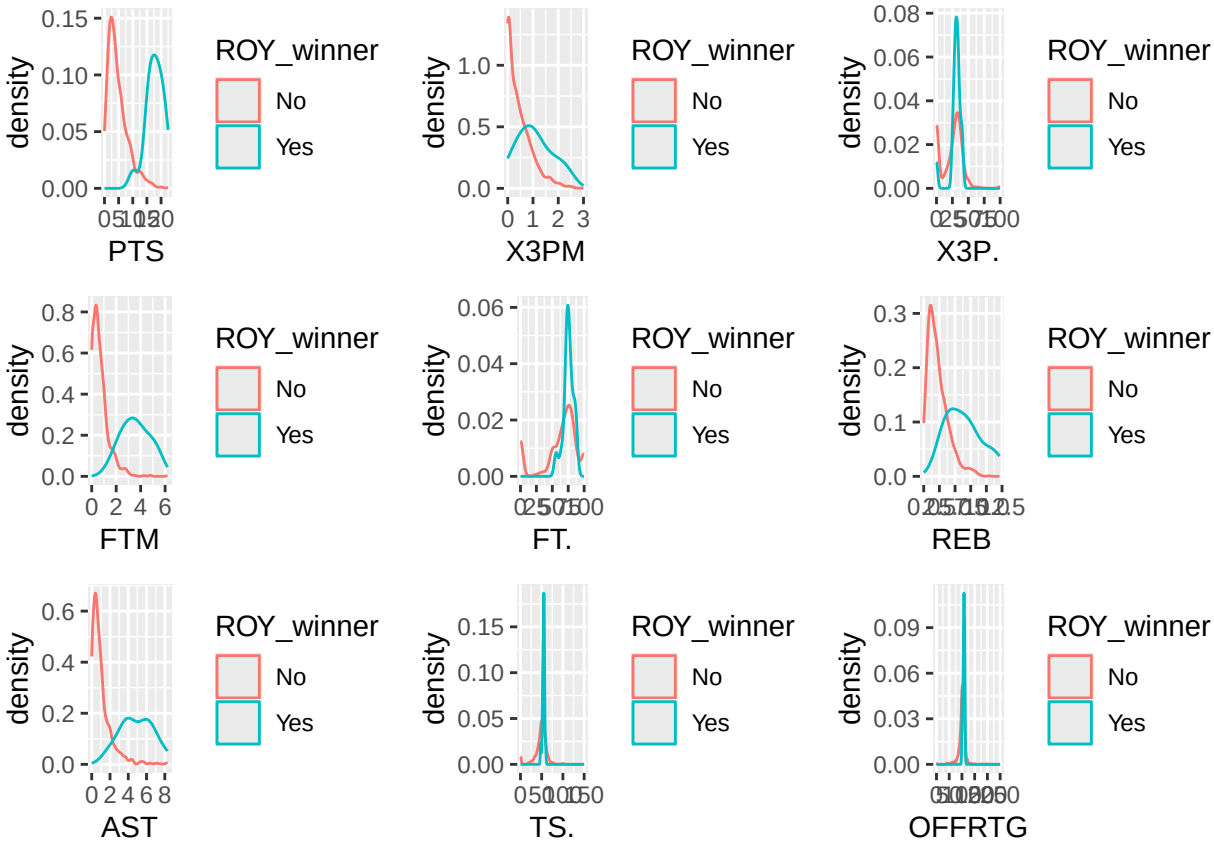
```
## After removing Double-Doubles & Triple-Doubles, we now have 25 predictors:
```

```r
names(prev_full)[-18]
```

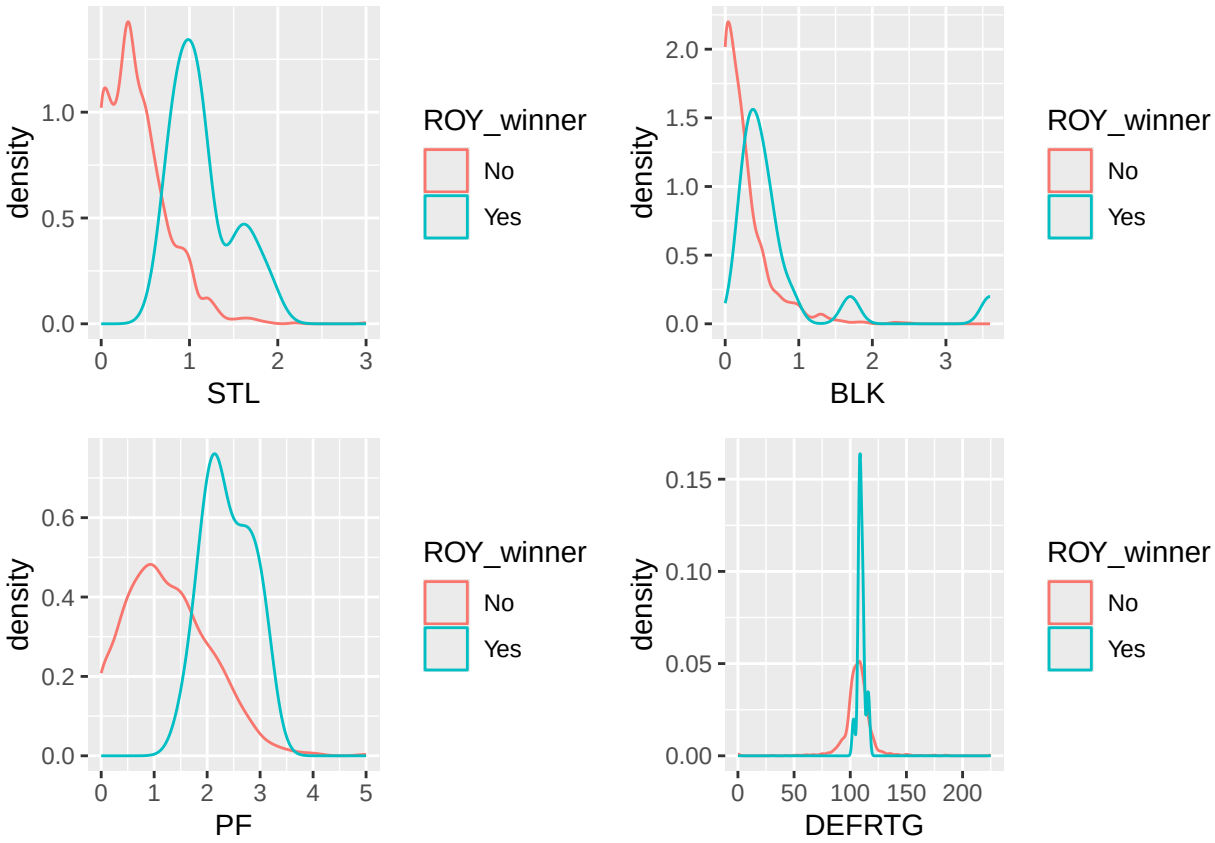
```
##  [1] "Player" "Team"   "Age"    "PTS"    "X3PM"   "X3P."   "FTM"    "FT."
##  [9] "REB"    "AST"    "TOV"    "STL"    "BLK"    "PF"     "X..."   "SEASON"
## [17] "PCT_GP" "OFFRTG" "DEFRTG" "NETRTG" "TS."    "USG."   "PACE"   "PIE"
## [25] "POSS"
```

Data is heavily concentrated at/near 0 for Double-Doubles and Triple-Doubles, therefore we will not find these variables particularly helpful in some of our models.

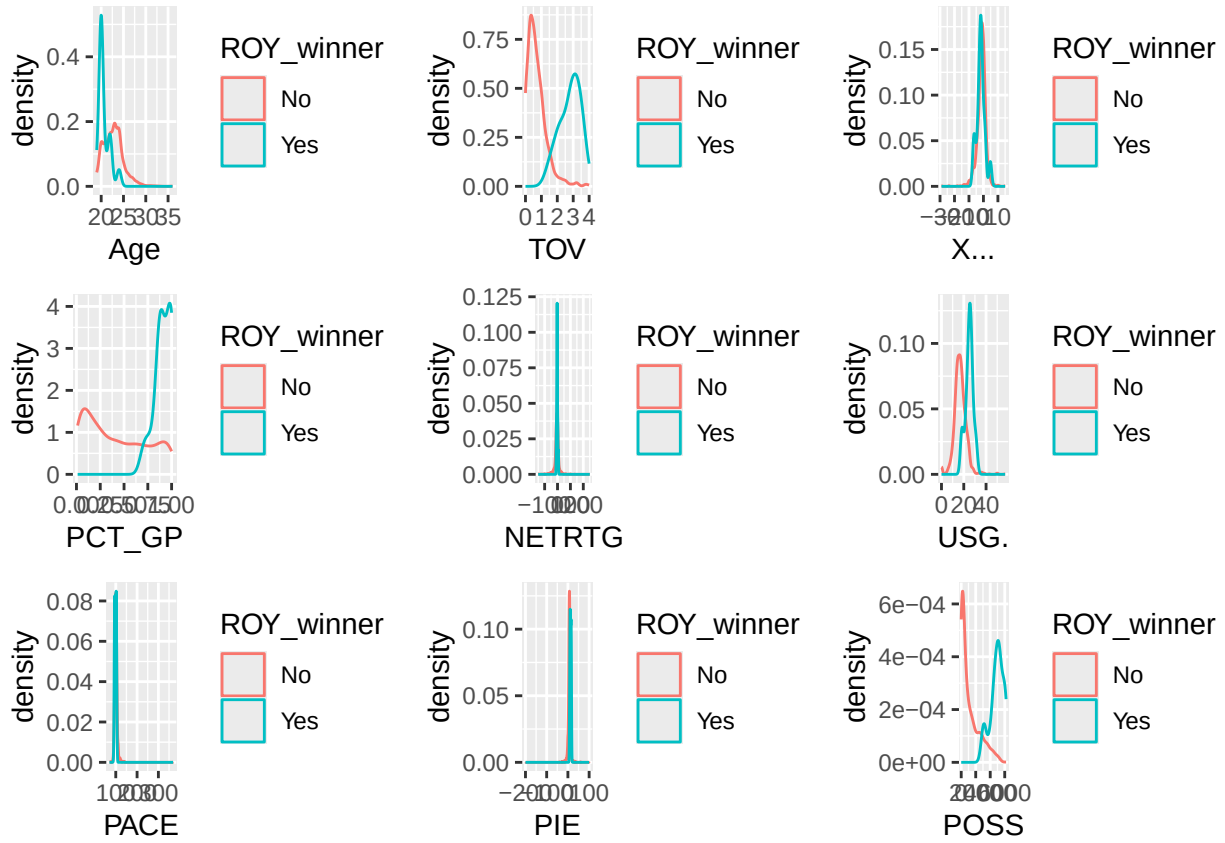
```r
g1 <- ggplot(prev_full, aes(x = PTS, color = ROY_winner)) + geom_density()
g2 <- ggplot(prev_full, aes(x = X3PM, color = ROY_winner)) + geom_density()
g3 <- ggplot(prev_full, aes(x = X3P., color = ROY_winner)) + geom_density()
g4 <- ggplot(prev_full, aes(x = FTM, color = ROY_winner)) + geom_density()
g5 <- ggplot(prev_full, aes(x = FT., color = ROY_winner)) + geom_density()
g6 <- ggplot(prev_full, aes(x = REB, color = ROY_winner)) + geom_density()
g7 <- ggplot(prev_full, aes(x = AST, color = ROY_winner)) + geom_density()
g8 <- ggplot(prev_full, aes(x = TS., color = ROY_winner)) + geom_density()
g9 <- ggplot(prev_full, aes(x = OFFRTG, color = ROY_winner)) + geom_density()

grid.arrange(g1, g2, g3, g4, g5, g6, g7, g8, g9, nrow = 3, ncol = 3)
```




```
g10 <- ggplot(prev_full, aes(x = STL, color = ROY_winner)) + geom_density()
g11 <- ggplot(prev_full, aes(x = BLK, color = ROY_winner)) + geom_density()
g12 <- ggplot(prev_full, aes(x = PF, color = ROY_winner)) + geom_density()
g13 <- ggplot(prev_full, aes(x = DEFRTG, color = ROY_winner)) + geom_density()

grid.arrange(g10, g11, g12, g13, nrow = 2)
```




```
g14 <- ggplot(prev_full, aes(x = Age, color = ROY_winner)) + geom_density()
g15 <- ggplot(prev_full, aes(x = TOV, color = ROY_winner)) + geom_density()
g16 <- ggplot(prev_full, aes(x = X..., color = ROY_winner)) + geom_density()
g17 <- ggplot(prev_full, aes(x = PCT_GP, color = ROY_winner)) + geom_density()
g18 <- ggplot(prev_full, aes(x = NETRTG, color = ROY_winner)) + geom_density()
g19 <- ggplot(prev_full, aes(x = USG., color = ROY_winner)) + geom_density()
g20 <- ggplot(prev_full, aes(x = PACE, color = ROY_winner)) + geom_density()
g21 <- ggplot(prev_full, aes(x = PIE, color = ROY_winner)) + geom_density()
g22 <- ggplot(prev_full, aes(x = POSS, color = ROY_winner)) + geom_density()

grid.arrange(g14, g15, g16, g17, g18, g19, g20, g21, g22, nrow = 3)
```



From these graphs, we expect the variables with the most separation in their density plots to be more impactful in predicting ROY (e.g. PTS, FTM, REB, AST, STL, PF, TOV, PCT_GP, USG., POSS).

BUILDING MODEL

```
#Split prev_full into 80% training & 20% testing data
#Ensure training/testing dataset has at least one ROY
set.seed(67)
train_index <- createDataPartition(prev_full$ROY_winner, p = 0.8, list = FALSE)
train <- prev_full[train_index, ]
test <- prev_full[-train_index, ]
```

```
#Create weights for modelling since ROTY winners are rare
#Utilized ChatGPT to calculate the weights
table(train$ROY_winner)
```

```
##
##   No  Yes
## 1148   13
```

```
y <- ifelse(train$ROY_winner == "Yes", 1, 0)
n <- length(y)
n1 <- sum(y == 1)
```

```r
n0 <- sum(y == 0)

weights <- ifelse(y == 1, n / (2 * n1), n / (2 * n0))

unique(weights)
```

```
## [1]  0.505662 44.653846
```

Lasso Regression Model

Logistic Regression fails due to perfect separation of the response variable. Therefore, we will push the data through a Lasso Regression Model to handle perfect separation & perform feature selection.

```r
#Scale numeric columns
num_cols <- sapply(train, is.numeric)
scaled_train <- train
scaled_train[, num_cols] <- scale(train[, num_cols], center = TRUE, scale = TRUE)
train_mean <- attr(scale(train[, num_cols]), "scaled:center")
train_sd   <- attr(scale(train[, num_cols]), "scaled:scale")

#Build the model
x <- model.matrix(ROY_winner ~ . -SEASON -Player -Team, data = scaled_train)[,-1]
y <- scaled_train$ROY_winner
lasso <- cv.glmnet(x, y, family = "binomial", alpha = 1, nfolds = 10,  weights = ifelse(y == "Yes", 44.6

coef(lasso, s = "lambda.min")
```

```
## 23 x 1 sparse Matrix of class "dgCMatrix"
##              lambda.min
## (Intercept) -3.56113240
## Age          .
## PTS          .
## X3PM         .
## X3P.         .
## FTM          0.56432583
## FT.          .
## REB          .
## AST          0.76476503
## TOV          .
## STL          0.05283835
## BLK          .
## PF           .
## X...         .
## PCT_GP       .
## OFFRTG       .
## DEFRTG       .
## NETRTG       .
## TS.          .
## USG.         .
## PACE         .
## PIE          .
## POSS         0.67722572
```

We observe that the most important variables in our lasso regression model were FTM, AST, STL, and POSS (interestingly).

```
#Find the best cutoff threshold for classification
pred_probs <- predict(lasso, newx = x, s = "lambda.min", type = "response")
pred_obj <- prediction(pred_probs, y)
eval <- performance(pred_obj, "acc")
best <- which.max(slot(eval, "y.values")[[1]])
cutoff <- slot(eval, "x.values")[[1]][best]
cat("The best cutoff threshold value is", cutoff, ".\n")
```

```
## The best cutoff threshold value is 0.9822166 .
```

```
#Create a confusion matrix using the new cutoff threshold
pred_class <- ifelse(pred_probs > 0.9822166, 1, 0)
table(Predicted = pred_class, Actual = y)
```

```
##          Actual
## Predicted   No  Yes
##         0 1146    7
##         1    2    6
```

Above is the confusion matrix when predicting ROTYs using the training data. The model seems to do OK, but there are some signs of underfitting (hinting towards increasing complexity in our future models).

```
scaled_test <- test
scaled_test[, num_cols] <- scale(test[, num_cols], center = train_mean, scale = train_sd)
x_test <- model.matrix(ROY_winner ~ . -SEASON -Player -Team, data = scaled_test)[, -1]

y_test <- scaled_test$ROY_winner

pred_prob <- predict(lasso, newx = x_test, s = "lambda.min", type = "response")
pred_class <- ifelse(pred_prob > 0.9822166, 1, 0)
table(Predicted = pred_class, Actual = y_test)
```

```
##          Actual
## Predicted  No Yes
##         0 286   3
```

We see that the model is not that precise when it comes to the testing dataset. This is due to the fact that ROTY winners are a relatively rare case. We observe that the lasso model missed all of the true positives (ROTYs).

For clarity, lets display the probabilities in a dataframe with the player, team, season, and whether or not they won the ROTY award.

```
prob_df <- as.data.frame(pred_prob)
merged_df <- cbind(
  test[, c("Player", "Team", "SEASON", "ROY_winner")],
  prob_df
)
```

```r
#Rank by highest to lowest probability to win ROTY, displaying up until all 3 ROTY are shown
merged_df <- merged_df[order(-merged_df$lambda.min), ]
head(merged_df, 10)
```

```
##                  Player Team    SEASON ROY_winner lambda.min
## 66         Kyrie.Irving  CLE 2011-2012        Yes  0.8935876
## 344    Donovan.Mitchell  UTA 2017-2018         No  0.8774858
## 304          Trey.Burke  UTA 2013-2014         No  0.8172153
## 235      Keyonte.George  UTA 2023-2024         No  0.7933215
## 192      Scottie.Barnes  TOR 2021-2022        Yes  0.7460150
## 61     DeMarcus.Cousins  SAC 2010-2011         No  0.7451950
## 717       Isaiah.Thomas  SAC 2011-2012         No  0.6628587
## 48       Bub.Carrington  WAS 2024-2025         No  0.6514219
## 1286     Zavier.Simpson  OKC 2021-2022         No  0.6399590
## 90   Karl.Anthony.Towns  MIN 2015-2016        Yes  0.6312501
```

All were displayed players were finalists or won ROTY except Keyonte George, Isaiah Thomas, Bub Carrington, & Zaviar Simpson.

Random Forest

Random Forest is more complex than lasso regression, and lasso regression tends to miss non-linear relationships, therefore we will try for a non-parametric model to try and predict ROTY: Random Forest.

Random forest is more robust to multicollinearity, therefore I will add all the predictors we removed for logistic/lasso regression.

```r
#Merge data
prev_full <- prev_rooks %>%
  left_join(prev_rooks_adv, by = c("Player" = "PLAYER", "Team" = "TEAM"))

#Split prev_full into 80% training & 20% testing data
#Ensure training/testing dataset has at least one ROY
set.seed(67)
train_index <- createDataPartition(prev_full$ROY_winner, p = 0.8, list = FALSE)
train <- prev_full[train_index, ]
test <- prev_full[-train_index, ]
```

```r
#Build the model, factoring in class imbalances within the training dataset
set.seed(67)
rf_model <- randomForest(
  ROY_winner ~ . -SEASON -Player -Team,
  data = train,
  ntree = 500,
  mtry = 4,
  importance = TRUE,
  classwt = c("No" = 0.505662, "Yes" = 44.653846),
  sampsize = c("No" = 13, "Yes" = 13)
)

rf_model
```

```
##
## Call:
##  randomForest(formula = ROY_winner ~ . - SEASON - Player - Team,      data = train, ntree = 500, mtry
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 4
##
##         OOB estimate of  error rate: 5.08%
## Confusion matrix:
##       No Yes class.error
## No  1090  58  0.05052265
## Yes    1  12  0.07692308
```

```
#Assume a threshold of 0.82
prob <- predict(rf_model, newdata = train, type = "prob")
pred <- ifelse(prob[, 2] > 0.82, "Yes", "No")
table(pred, train$ROY_winner)
```

```
##
## pred    No  Yes
##   No  1138    1
##   Yes   10   12
```

```
prob <- predict(rf_model, newdata = test, type = "prob")
pred <- ifelse(prob[, "Yes"] > 0.82, "Yes", "No")
table(pred, test$ROY_winner)
```

```
##
## pred   No Yes
##   No  284   2
##   Yes   2   1
```

We see that the model is relatively better than lasso regression at predicting ROTY on unseen data. Now, our model is better at finding the true positives within our testing set, however, we see this model over classifying ROTY winners (false positives).

(It is important to note that these thresholds are calculated intuition and what best fits the training data. Therefore, the comparison between lasso regression and random forest may be misinterpreted via this way)

For clarity, lets display the probabilities in a dataframe with the player, team, season, and whether or not they won the ROTY award.

```
prob_df <- as.data.frame(prob)
merged_df <- cbind(
  test[, c("Player", "Team", "SEASON", "ROY_winner")],
  prob_df
)

#Rank by highest to lowest probability to win ROTY, displaying up until all 3 ROTY are shown
merged_df <- merged_df[order(-merged_df$Yes), ]
head(merged_df[, -5], 5)
```

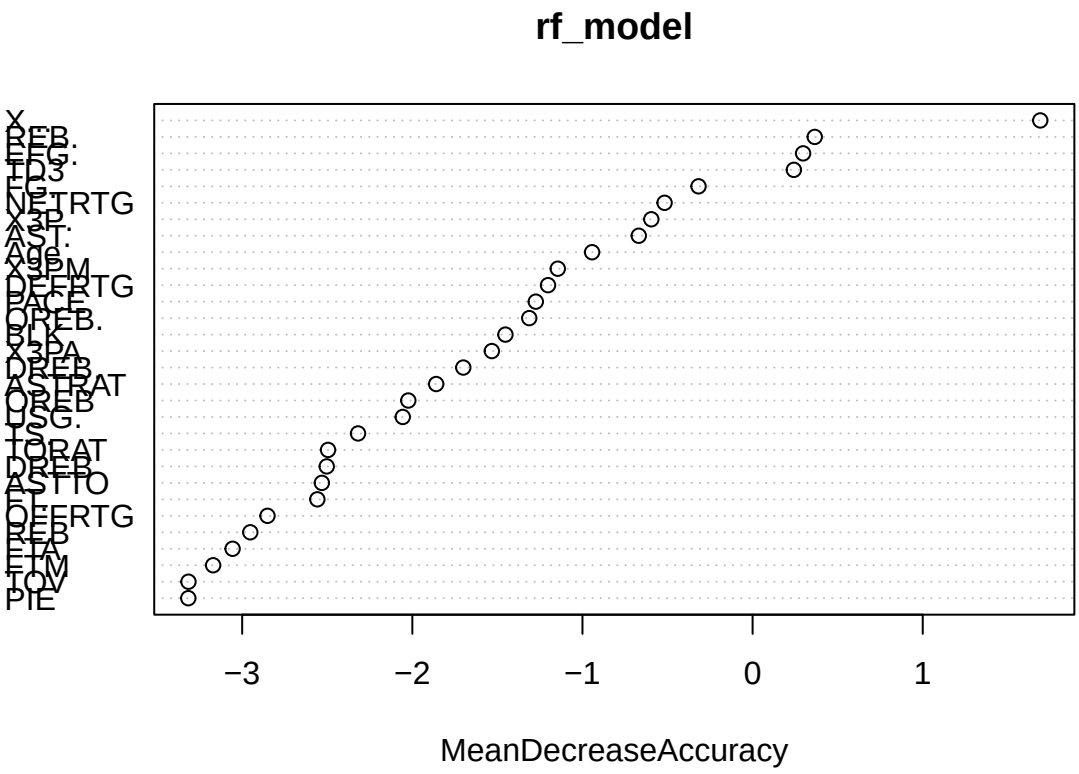
```
##                 Player Team    SEASON ROY_winner   Yes
```

```
## 344   Donovan.Mitchell  UTA 2017-2018         No 0.868
## 192     Scottie.Barnes  TOR 2021-2022        Yes 0.860
## 61    DeMarcus.Cousins  SAC 2010-2011         No 0.838
## 90  Karl.Anthony.Towns  MIN 2015-2016        Yes 0.796
## 66         Kyrie.Irving  CLE 2011-2012        Yes 0.776
```

All were displayed players were finalists or won ROTY with relatively high probabilities.

To attempt to make the model more interpretable & simpler, we will sacrifice some bias to decrease the variance of the model by only choosing variables that are the most important in the model.

```
varImpPlot(rf_model, type = 1)
```




```
imp <- importance(rf_model)
selected_vars <- rownames(imp)[order(imp[,"MeanDecreaseAccuracy"], decreasing = TRUE)][1:20]
cat("From the Variable Importance Plot, it seems that the top 20 variables that contribute the most towa
```

```
## From the Variable Importance Plot, it seems that the top 20 variables that contribute the most toward
```

```
selected_vars
```

```
##  [1] "X..."    "REB."    "EFG."    "TD3"     "FG."     "NETRTG" "X3P."    "AST."
##  [9] "Age"     "X3PM"    "DEFRTG"  "PACE"    "OREB."   "BLK"    "X3PA"    "DREB."
## [17] "ASTRAT"  "OREB"    "USG."    "TS."
```

Random Forest (Tuned)

```
#Build the model based on the Variable Importance Plot
formula <- as.formula(paste("ROY_winner ~", paste(selected_vars, collapse = " + ")))

set.seed(67)
rf_tuned <- randomForest(
  formula,
  data = train,
  ntree = 500,
  mtry = 4,
  importance = TRUE,
  classwt = c("No" = 0.505662, "Yes" = 44.653846),
  sampsize = c("No" = 13, "Yes" = 13)
)

rf_tuned
```

```
##
## Call:
##  randomForest(formula = formula, data = train, ntree = 500, mtry = 4,      importance = TRUE, classw
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 4
##
##         OOB estimate of  error rate: 4.48%
## Confusion matrix:
##       No Yes class.error
## No  1101  47  0.04094077
## Yes    5   8  0.38461538
```

```
#Assume a threshold of 0.82
pred_tuned <- predict(rf_tuned, newdata = train, type = "prob")
tuned_prob <- ifelse(pred_tuned[, "Yes"] > 0.74, "Yes", "No")
table(tuned_prob, train$ROY_winner)
```

```
##
## tuned_prob   No  Yes
##        No  1144    1
##        Yes    4   12
```

```
pred_tuned <- predict(rf_tuned, newdata = test, type = "prob")
tuned_prob <- ifelse(pred_tuned[, "Yes"] > 0.74, "Yes", "No")
table(tuned_prob, test$ROY_winner)
```

```
##
## tuned_prob  No Yes
##        No  284   2
##        Yes   2   1
```

The reduced model seems to do better than the full model when it comes to the training data (unseen), but similar when it comes to the testing data. However, the reduced model is more interpretable than the full model. This indicates some overfitting within this reduced model.

```r
prob_df <- as.data.frame(pred_tuned)
merged_df <- cbind(
  test[, c("Player", "Team", "SEASON", "ROY_winner")],
  prob_df
)

#Rank by highest to lowest probability to win ROTY, displaying up until all 3 ROTY are shown
merged_df <- merged_df[order(-merged_df$Yes), ]
head(merged_df[-5], 13)
```

```
##                       Player Team    SEASON ROY_winner   Yes
## 66              Kyrie.Irving  CLE 2011-2012        Yes 0.912
## 773          Jordan.Clarkson  LAL 2014-2015         No 0.868
## 344         Donovan.Mitchell  UTA 2017-2018         No 0.756
## 139 Shai.Gilgeous.Alexander  LAC 2018-2019         No 0.732
## 360          Harry.Giles.III  SAC 2018-2019         No 0.704
## 192           Scottie.Barnes  TOR 2021-2022        Yes 0.670
## 433           Brandon.Miller  CHA 2023-2024         No 0.608
## 61          DeMarcus.Cousins  SAC 2010-2011         No 0.590
## 564               Kyle.Kuzma  LAL 2017-2018         No 0.588
## 370          P.J..Washington  CHA 2019-2020         No 0.582
## 104             Jamal.Murray  DEN 2016-2017         No 0.562
## 159                 Naz.Reid  MIN 2019-2020         No 0.558
## 90        Karl.Anthony.Towns  MIN 2015-2016        Yes 0.546
```

We can see that the number of observations it takes for us to see all 3 ROTY is 10, with Karl Anthony-Towns having a 65% chance of winning ROTY (surprising since he was unanomously voted). The only players in this list who were not ROTY or were not runners up for ROTY were Jordan Clarkson, Shai Gilgeous-Alexander, and Trey Burke.

Random Forest (Tuned & Cross-validated)

```r
#Build the model based on the Variable Importance Plot
#Use 5-fold cross-validation to reduce overfitting
ctrl <- trainControl(method = "cv", number = 5)

set.seed(67)
rf_cv <- train(
  formula,
  data = train,
  method = "rf",
  trControl = ctrl,
  tuneGrid = data.frame(mtry = 2:10),
  ntree = 500,
  classwt = c("No" = 0.505662, "Yes" = 44.653846),
  sampsize = c("No" = 10, "Yes" = 10)
)

rf_cv
```

```
## Random Forest
##
## 1161 samples
```

```
##   20 predictor
##    2 classes: 'No', 'Yes'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 930, 929, 929, 928, 928
## Resampling results across tuning parameters:
##
##   mtry  Accuracy   Kappa
##    2    0.9612399  0.2608593
##    3    0.9543396  0.2411628
##    4    0.9500441  0.2191085
##    5    0.9509173  0.2185927
##    6    0.9457597  0.2168732
##    7    0.9483459  0.2256225
##    8    0.9414492  0.2023759
##    9    0.9379936  0.1803718
##   10    0.9474801  0.2208363
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 2.
```

```
#Assume a threshold of 0.82
pred_cv <- predict(rf_cv, newdata = train, type = "prob")
cv_prob <- ifelse(pred_cv[, "Yes"] > 0.83, "Yes", "No")
table(cv_prob, train$ROY_winner)
```

```
##
## cv_prob   No  Yes
##     No  1148    5
##     Yes    0    8
```

```
pred_cv <- predict(rf_cv, newdata = test, type = "prob")
cv_prob <- ifelse(pred_cv[, "Yes"] > 0.83, "Yes", "No")
table(cv_prob, test$ROY_winner)
```

```
##
## cv_prob  No Yes
##      No 286   3
```

The cross-validated model seems to do worse than the full model and reduced model when it comes to the testing data (unseen), but does slightly better on the training data in comparison to the full model. However, the cross-validated model is more interpretable & more consistent than the full model.

```
prob_df <- as.data.frame(pred_cv)
merged_df <- cbind(
  test[, c("Player", "Team", "SEASON", "ROY_winner")],
  prob_df
)

#Rank by highest to lowest probability to win ROTY, displaying up until all 3 ROTY are shown
merged_df <- merged_df[order(-merged_df$Yes), ]
head(merged_df[-5], 11)
```

```
##                      Player Team    SEASON ROY_winner   Yes
## 66             Kyrie.Irving  CLE 2011-2012        Yes 0.830
## 773         Jordan.Clarkson  LAL 2014-2015         No 0.768
## 344        Donovan.Mitchell  UTA 2017-2018         No 0.668
## 139 Shai.Gilgeous.Alexander  LAC 2018-2019         No 0.658
## 192          Scottie.Barnes  TOR 2021-2022        Yes 0.642
## 564              Kyle.Kuzma  LAL 2017-2018         No 0.634
## 360         Harry.Giles.III  SAC 2018-2019         No 0.602
## 433          Brandon.Miller  CHA 2023-2024         No 0.596
## 370         P.J..Washington  CHA 2019-2020         No 0.570
## 104            Jamal.Murray  DEN 2016-2017         No 0.558
## 90       Karl.Anthony.Towns  MIN 2015-2016        Yes 0.530
```

We can see that the number of observations it takes for us to see all 3 ROTY is 10, with Karl Anthony-Towns having a 65% chance of winning ROTY (surprising since he was unanomously voted). The only players in this list who were not ROTY or were not runners up for ROTY were Jordan Clarkson, Shai Gilgeous-Alexander, Kyle Kuzma, Harry Giles III, P.J Washington, and Jamal Murray.

TESTING USING CURRENT ROOKIES

```
#Remove all variables from the testing dataset as we did with the training dataset
current_full <- current_full %>%
  mutate(across(where(is.character), as.factor)) %>%
  select(
    -c(
      GP,          # Corresponds to PCT_GP
      OREB, DREB, # Linear combination of REB
      AST., ASTTO, ASTRAT, # Corresponds to AST
      OREB., DREB., # Linear combination of REB.
      REB.,        # Corresponds to REB
      TORAT,       # Corresponds to TOV
      Min, FGA, X3PA, FTA, FP,
      EFG., W, L, FG., FGM, DD2, TD3, TM_GP
    )
  )
```

Lasso Model

```
lasso_current <- current_full %>%
  select(-SEASON)

#Scale numeric columns
num_cols <- sapply(lasso_current, is.numeric)

scaled_current <- lasso_current
scaled_current[, num_cols] <- scale(lasso_current[, num_cols], center = train_mean, scale = train_sd)

x_current <- model.matrix(~ . -Player -Team, data = scaled_current)[,-1]
pred_prob <- predict(lasso, newx = x_current, s = "lambda.min", type = "response")
```

```r
#Show the top 10 rookies with the highest probability of become ROTY
df <- data.frame(player_name = lasso_current$Player, probability = pred_prob)
df <- df[order(-df$lambda.min), ]
head(df, 5)
```

```
##             player_name lambda.min
## 19      DaRon Holmes II          1
## 31        Myron Gardner          1
## 41           Noah Penda          1
## 58 Collin Murray-Boyles          1
## 40     Brooks Barnhizer          1
```

```r
tail(df, 5)
```

```
##         player_name  lambda.min
## 26      Adou Thiero 0.005875551
## 37 Hunter Dickinson 0.005724165
## 45    Hunter Sallis 0.005625288
## 13     Noa Essengue 0.004287773
## 46        Koby Brea 0.004279281
```

We can observe that the probability that each player in the 2025-2026 rookie class wins ROTY is either really high (100%) or really low (<0.1%) for our lasso regression model. Therefore, we will not go through with this model.

Random Forest Full Model

```r
current_full <- current_rooks %>%
  left_join(current_rooks_adv, by = c("Player" = "PLAYER", "Team" = "TEAM"))
```

```r
prob <- predict(rf_model, newdata = current_full, type = "prob")[, "Yes"]
df <- data.frame(player_name = current_full$Player, probability = prob)
df <- df[order(-df$probability), ]
head(df, 10)
```

```
##         player_name probability
## 15     Cooper Flagg       0.644
## 9      Kon Knueppel       0.588
## 43     VJ Edgecombe       0.572
## 34   Jeremiah Fears       0.556
## 35      Derik Queen       0.496
## 28    Cedric Coward       0.444
## 55     Dylan Harper       0.430
## 10 Ryan Kalkbrenner       0.254
## 59       Ace Bailey       0.182
## 61      Tre Johnson       0.180
```

```r
write.csv(df, "predictions.csv", row.names = FALSE)
```

All players correspond to the Kia Rookie Ladder list (except for Dylan Harper). Probabilities seem more or less expected. Surprising to see Cedric Coward (3rd on the Kia Rookie Ladder as of November 26th) having only the 6th highest probability of winning ROTY.

Random Forest Reduced Model

```r
prob <- predict(rf_tuned, newdata = current_full, type = "prob")[, "Yes"]
df <- data.frame(player_name = current_full$Player, probability = prob)
df <- df[order(-df$probability), ]
head(df, 10)
```

```
##                 player_name probability
## 35              Derik Queen       0.816
## 15             Cooper Flagg       0.702
## 34           Jeremiah Fears       0.658
## 55             Dylan Harper       0.636
## 28            Cedric Coward       0.580
## 43             VJ Edgecombe       0.574
## 9              Kon Knueppel       0.502
## 56       David Jones Garcia       0.492
## 59               Ace Bailey       0.442
## 24 Yanic Konan Niederhauser       0.434
```

Surprising to see Derik Queen having a very high probability to win ROTY (~80%).

Random Forest Reduced & Cross-Validated Model

```r
prob <- predict(rf_cv, newdata = current_full, type = "prob")[, "Yes"]
df <- data.frame(player_name = current_full$Player, probability = prob)
df <- df[order(-df$probability), ]
head(df, 10)
```

```
##                 player_name probability
## 35              Derik Queen       0.758
## 15             Cooper Flagg       0.688
## 55             Dylan Harper       0.638
## 34           Jeremiah Fears       0.634
## 43             VJ Edgecombe       0.534
## 28            Cedric Coward       0.528
## 9              Kon Knueppel       0.518
## 59               Ace Bailey       0.438
## 56       David Jones Garcia       0.382
## 24 Yanic Konan Niederhauser       0.366
```

Throughout the reduced and cross-validated Random Forest models, we see Derik Queen top the charts when it comes to winning ROTY (though by a relatively small margin). What is surprising is that Kon Knueppel

and Cedric Coward are relatively lower compared to the other top 10 players with the highest probabilities. However, it seems that the full Random Forest model seems to do the best when it comes to unseen data. The reduced and cross-validated models became too simple for the data. By decreasing the variance, we face the bias-variance tradeoff, therefore we increase bias and weaken our models ability to capture non-linear relationships with reduced models.

Due to the previous findings I believe that the Random Forest Full Model does the best job at predicting ROTY on unseen data (at the expense of interpretabiliity).

Sources:
- OpenAI ChatGPT (2025). Used for guidance on code organization.